

19-technical-interview

Modul 19: Technical Interview — Persiapan Kerja & Magang untuk Lulusan SMK RPL

Level: □ Intermediate

Jam: 12 (3 sesi × 4 jam)

Prasyarat: Modul 01-18 (atau setara) — sudah punya proyek/portofolio

Output: Mock interview video + CV ATS-ready + Portfolio final

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Menjawab pertanyaan behavioral dengan struktur STAR yang meyakinkan
- Membuat CV dan LinkedIn yang lolos filter ATS & menarik recruiter
- Menyelesaikan soal DSA coding dengan 5 pola utama (sliding window, two pointers, fast/slow pointer, BFS/DFS, DP)
- Menjawab system design interview dengan metode 1-2-3 (klarifikasi → high-level → deep dive)
- Menampilkan portfolio proyek secara profesional dan terstruktur
- Melakukan mock interview dengan percaya diri
- Menegosiasikan tawaran kerja dan melakukan follow-up profesional

Materi

Sesi	Topik
1	Behavioral Interview & CV Preparation — STAR method, CV ATS-friendly, LinkedIn, portfolio
2	DSA Patterns & System Design Interview — 5 pola coding, metode 1-2-3, whiteboard app

Sesi Topik

3 Mock Interview & Portfolio Finalization — Simulasi, coding strategy, negotiation, follow-up

Output Akhir Modul

Mock Interview Video — rekaman dirimu sendiri menjawab behavioral + coding + system design (30 menit)

CV ATS-Ready — PDF dengan format yang lolos filter otomatis

Portfolio Final — GitHub + live demo + project walk-through siap cerita

Ringkasan: Behavioral Interview — Metode STAR

Wawancara perilaku bobotnya setara dengan coding. Gunakan STAR.

Huruf	Arti	Contoh
Situation	Konteks	"Dalam proyek kelompok akhir semester..."
Task	Tanggung jawab	"Saya bertugas bangun REST API dan integrasi database."
Action	Langkah konkret	"Saya desain skema, buat endpoint dengan Express, tulis unit tes"
Result	Dampak	"API selesai 3 hari lebih cepat. Nilai proyek A."

Pertanyaan umum: "Ceritakan kegagalan terbesar." / "Pernah konflik tim?" / "Kenapa kami harus hire kamu?"

→ Siapkan 5 jawaban STAR. Rekam pakai HP. Evaluasi.

Ringkasan: CV & LinkedIn

Area	Checklist
CV	PDF simpel (ATS-friendly), keyword dari job description, 2-3 proyek dengan link
LinkedIn	Foto profesional, headline jelas, about section, skills + endorsement
Portfolio site	Hero, tentang, proyek (card + link), skill, kontak

Kesalahan fatal CV: typo nama perusahaan, email tidak profesional, klaim skill tidak relevan.

Ringkasan: DSA Coding — 5 Pola Utama

Pola	Kapan Pakai	Kompleksitas Khas
Sliding Window	Subarray/substring optimal (maks/min)	$O(n)$
Two Pointers	Array terurut, cari pasangan	$O(n)$
Fast & Slow Pointer	Linked list cycle, middle node	$O(n)$
BFS/DFS	Pencarian jalur, level order, shortest path	$O(V+E)$
DP	Overlapping subproblems, min/max/count	$O(n)$ / $O(n^2)$

Tips: NeetCode 150. 20-30 menit/soal. Jika stuck: lihat solusi, pahami, ulangi 2 hari kemudian.

Ringkasan: System Design — Metode 1-2-3

Langkah	Durasi	Aktivitas
1. Klarifikasi	3-5 menit	Tanya kebutuhan, pengguna, prioritas
2. High-Level	5-7 menit	Gambar arsitektur + komponen utama
3. Deep Dive	10-15 menit	DB schema, API design, data flow

Contoh soal untuk fresh grad: URL shortener, chat app, parking lot, todo API, rating system.

Ringkasan: Live Coding Strategy

1. Baca soal 2x → tanya clarifying questions → buat contoh
2. Sebut brute force + kompleksitas
3. Brainstorm optimasi → tulis pseudocode
4. Koding — syntax rapi, variable jelas
5. Test dengan contoh + edge cases
6. Sebut kompleksitas akhir + trade-off

Paling penting: *Think out loud sepanjang waktu.*

Ringkasan: Portofolio Walkthrough

Struktur 5 menit: judul → tech stack → arsitektur → fitur utama → tantangan → pelajaran.

Checklist final:

- ☐ README proyek lengkap (stack, fitur, screenshot, cara jalanin)
- ☐ Live demo bisa diakses (Vercel/Netlify/Railway)
- ☐ GitHub rapi (commit message, .gitignore, repositori bersih)

Ringkasan: Mock Interview, Negosiasi & Follow-Up

Tahap	Tindakan
Mock	Latihan 30 menit dengan teman. Evaluasi pakai tabel skor.
Negosiasi	Gaji, training, perangkat — riset dulu, bawa data.
Follow-up	Thank you email (1 jam) → follow-up (3 hari) → feedback (jika ditolak).

AI Prompt Exercises

Sepanjang modul, latihan pakai AI:

- "Generate 5 behavioral interview questions with STAR answer templates for a fresh grad SMK RPL student."
- "Review my CV text. Give feedback on ATS compatibility, keyword density, and formatting."
- "Give me a LeetCode Easy coding problem. I'll answer it. Then critique my solution — complexity, edge cases, code style."
- "Roleplay as a tech interviewer. Ask me system design questions for a fresh grad position."
- "Simulate a mock interview. Ask me 3 behavioral questions, 1 coding question, and 1 system design question. Give me scores and feedback."
- "Write a thank-you email template after a technical interview at a startup."
- "Evaluate my project walkthrough script. What's missing? What can be improved?"
- "Create a negotiation script for a fresh grad offer in Jakarta. The offered salary is 4 million IDR."

Referensi:

- [NeetCode Roadmap](#)
- *Cracking the Coding Interview* — Gayle Laakmann McDowell
- *System Design Interview* — Alex Xu
- [STAR Method Guide — themuse.com](#)
- Canva / Overleaf untuk template CV

1. Behavioral Interview & CV Preparation

Target: Kuasai metode STAR, siapkan CV ATS-friendly, optimasi LinkedIn, bangun portfolio site.

Metode STAR

STAR adalah struktur jawaban untuk pertanyaan behavioral. Interviewer ingin lihat **bukti konkret** — bukan teori.

Huruf	Arti	Fungsi
Situation	Konteks / latar belakang	Kasih gambaran situasi
Task	Tanggung jawab spesifik	Apa peran kamu
Action	Langkah konkret yang diambil	Yang kamu lakukan
Result	Dampak / hasil	Bukti keberhasilan

Contoh Penerapan STAR

Pertanyaan: "Ceritakan saat kamu menghadapi deadline ketat."

S: Proyek RPL semester 5 – bikin aplikasi perpustakaan dalam 2 minggu. Tim cuma 3 orang, satu anggota tiba-tiba sakit.

T: Saya harus menyelesaikan backend (REST API + database) plus membantu frontend yang ditinggal.

A: Saya prioritaskan fitur inti dulu – login, CRUD buku, peminjaman. Sisanya (laporan, filter) ditunda. Saya lembur 2 hari terakhir bantu frontend pake Tailwind.

R: Aplikasi selesai tepat waktu. Demo jalan lancar. Nilai A. Saya belajar prioritisasi fitur berdasarkan dampak.

Pertanyaan: "Ceritakan kegagalan terbesar."

S: Proyek akhir semester 4 – aplikasi kasir. Saya terlalu percaya diri pakai framework baru tanpa riset.

T: Saya lead developer, harus deliver fitur transaksi dalam 1 minggu.

A: Hari ke-3 saya sadar dokumentasi framework kurang. Saya ganti ke Express (yang saya kuasai). Rela begadang 2 minta ngejar fitur yang hilang.

R: Fitur transaksi jadi telat 1 hari, tapi tetap selesai. Saya belajar: eksperimen itu bagus, tapi pastikan ada fallback.

Pertanyaan Behavioral Umum

Pertanyaan	Yang Dinilai	STAR Siapkan
"Ceritakan tentang dirimu."	First impression, relevansi	30 detik: nama, skill utama,
"Kenapa kami harus hire kamu?"	Kepercayaan diri	Link skill kamu ke kebutuhan
"Apa kelemahan terbesar?"	Self-awareness	Kelemahan nyata + langkah
"Konflik dalam tim?"	Soft skill	Negosiasi, komunikasi, komp
"Proyek paling bangga?"	Technical depth	Detail arsitektur, tantangan,
"Mau kerja di mana dalam 5 tahun?"	Ambisi	Realistis, relevan dengan po
"Pernah dapat kritik pedas?"	Growth mindset	Terima, evaluasi, perbaiki
"Kerja tim vs mandiri, suka mana?"	Kolaborasi	Keduanya, tergantung situasi

Tips: Rekam jawaban pakai HP. Putar ulang. Jika ragu-ragu, ulangi sampai natural.

CV / Resume Tips

Format CV Fresh Grad RPL

Elemen	Wajib?	Keterangan
Nama & kontak	☐	Email profesional, LinkedIn, GitHub, no. HP
Pendidikan	☐	SMK RPL (tahun lulus), nilai jika > 85
Skill teknis	☐	Bahasa, framework, database, tools
Proyek	☐	2-3 proyek dengan link + deskripsi + stack
Sertifikasi	☐	BNSP, Dicoding, Coursera (relevan IT)
Organisasi / Lomba	☐	OSIS, LKS, hackathon — bukti soft skill
Foto	☐	Tidak wajib, hindari selfie kasual
Hobi	☐	Kecuali relevan (nulis blog teknis, kontribusi open source)

ATS-Friendly Tips

ATS (Applicant Tracking System) — software HR yang **filter CV otomatis**.

1. **PDF simpel** — Jangan pakai tabel, kolom, ikon, atau grafis. ATS baca teks linear.
2. **Keyword dari job description** — Baca deskripsi kerja, ambil skill yang diminta, pastikan muncul di CV.
3. **Format standar:** Nama → Kontak → Ringkasan (opsional) → Pendidikan → Skill → Proyek → Sertifikasi.

4. **Gunakan kata kunci baku:** "JavaScript", "React", "Node.js", "MySQL", "REST API" — bukan "bikin web", "otak atik kode".
5. **Simpan sebagai PDF** — Format tidak berantakan di device mana pun.

Kesalahan Fatal CV

Kesalahan	Dampak
Typo nama perusahaan	Langsung discard
Email tidak profesional (cowokkeren22@)	Kesan tidak serius
Skill tidak relevan (Microsoft Word untuk lamaran backend)	Waste space
Klaim expert padahal baru 1x pakai	Interview akan ketahuan
Tidak menyertakan link portfolio/GitHub	Bukti tidak ada

LinkedIn Profile Optimization

LinkedIn sering jadi **pencarian pertama** sebelum wawancara.

Checklist Profile

- ☐ Foto profesional (bukan selfi, pakaian rapi, latar netral)
- ☐ Headline jelas: *"SMK RPL Student | Backend Developer | React & Node.js Enthusiast"*
- ☐ About section: 3-4 kalimat — siapa kamu, skill utama, target karir
- ☐ Experience: proyek sekolah, magang, freelance (pakai bullet points)
- ☐ Skills: tambahkan 5-10 skill relevan, minta endorsement teman
- ☐ Sertifikasi: upload foto sertifikat
- ☐ Custom URL: `linkedin.com/in/namakamu`
- ☐ Posting minimal 1x minggu: proyek baru, belajar hal baru, sertifikasi

Contoh About Section

Backend developer lulusan SMK RPL 2025. Pengalaman membangun REST API dengan Node.js & Express, integrasi database MySQL, dan deploy ke Railway/Vercel. Active contributor di 2 proyek open source. Saat ini fokus belajar TypeScript dan PostgreSQL.

#backend #nodejs #react #mysql #smkrpl

Portfolio Site Structure

Portfolio website = bukti konkret. Tanpa portfolio, CV cuma klaim.

Struktur Minimal

Halaman	Isi
Home	Hero section — nama, role, CTA ke proyek
Tentang	2-3 paragraf: latar belakang, skill, minat
Proyek	Card grid: nama, screenshot, stack, link live + GitHub
Skill	Ikon + nama teknologi (bisa progress bar)
Kontak	Form atau link: Email, LinkedIn, GitHub

Tech Stack Rekomendasi

Level	Stack
Beginner	HTML + CSS + JS (deploy GitHub Pages)
Intermediate	React + Tailwind + Vercel
Advanced	Next.js + Framer Motion + Sanity CMS

Wajib Ada di Setiap Proyek

1. **Nama & deskripsi singkat** — apa yang dibuat dan kenapa.
2. **Screenshot / GIF demo** — bukti berjalan.
3. **Tech stack** — bahasa, framework, database.
4. **Link live demo** — Vercel, Netlify, GitHub Pages.
5. **Link GitHub** — recruiter bisa lihat kode.
6. **Fitur utama** — bullet points.

Aturan emas: Jika recruiter butuh lebih dari 2 klik untuk lihat proyekmu, mereka sudah pindah ke kandidat lain.

Latihan

1. **STAR Practice:** Ambil 3 pertanyaan behavioral dari tabel di atas. Tulis jawaban STAR lengkap (min 4 kalimat per huruf). Baca keras-keras sampai terdengar natural.
2. **CV Audit:** Buka CV kamu sekarang. Cek: (a) apakah ada typo? (b) apakah keyword sesuai job description? (c) apakah format PDF simpel/bisa dibaca ATS? Tuliskan 3 perbaikan.

3. **LinkedIn Makeover:** Ikuti semua checklist LinkedIn di atas. Screenshot sebelum-sesudah. Minta pendapat teman.
4. **Portfolio Site Wireframe:** Gambar (kertas/Figma) layout portfolio kamu. Tentukan: warna, font, halaman apa saja, tech stack. Tulis alasan setiap pilihan.

2. DSA Patterns & System Design Interview

Target: Kuasai 5 pola DSA utama + framework jawab system design dengan metode 1-2-3.

Common DSA Patterns

1. Sliding Window

Gunakan saat: Input array/string, cari subarray/substring optimal (maks/min/panjang tertentu).

Pola kunci:

- Fixed window — geser jendela ukuran tetap, catat nilai tiap posisi.
- Variable window — perluas right, ciutkan left saat kondisi rusak.

Substring terpanjang tanpa karakter berulang

```
def panjang_substring_unik(s: str) -> int:
    seen = set()
    left = max_len = 0
    for right, ch in enumerate(s):
        while ch in seen:
            seen.remove(s[left])
            left += 1
        seen.add(ch)
        max_len = max(max_len, right - left + 1)
    return max_len
```

Test

```
print(panjang_substring_unik("abcabcbb")) # 3 ("abc")
print(panjang_substring_unik("bbbbb"))   # 1 ("b")
print(panjang_substring_unik(""))         # 0
```

2. Two Pointers

Gunakan saat: Array sudah terurut atau perlu pasangan elemen.

Pola kunci:

- Pointer kiri + kanan, bergerak ke tengah.
- Atau slow/fast pointer satu arah.

Cari dua angka dalam array terurut yang jumlahnya = target

```
def dua_jumlah(arr, target):
    kiri, kanan = 0, len(arr) - 1
    while kiri < kanan:
        total = arr[kiri] + arr[kanan]
        if total == target:
            return [arr[kiri], arr[kanan]]
        elif total < target:
            kiri += 1
        else:
            kanan -= 1
    return []

# Test
print(dua_jumlah([1, 2, 3, 4, 6], 6)) # [2, 4]
print(dua_jumlah([1, 2, 3], 10))      # []
```

3. Fast & Slow Pointer (Floyd's Algorithm)

Gunakan saat: Linked list — deteksi siklus, cari titik tengah.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

Deteksi cycle dalam linked list

```
def ada_siklus(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow == fast:
        return True
    return False
```

Membuat linked list dengan cycle

```
node1 = ListNode(3)
node2 = ListNode(2)
```

```

node3 = ListNode(0)
node4 = ListNode(-4)
node1.next = node2
node2.next = node3
node3.next = node4
node4.next = node2 # cycle

print(ada_siklus(node1)) # True

```

4. BFS & DFS (Graph/Tree)

Gunakan saat: Pencarian jalur, level-order traversal, shortest path.

Algoritma	Struktur Data	Cocok
BFS	Queue	Shortest path di graph tak berbobot, level order
DFS	Stack (rekursif/eksplisit)	Semua kemungkinan jalur, topological sort

```

from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# BFS - level order traversal
def level_order(root):
    if not root:
        return []
    hasil, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        hasil.append(level)
    return hasil

# DFS - inorder traversal (rekursif)
def inorder(root):

```

```

    if not root:
        return []
    return inorder(root.left) + [root.val] + inorder(root.right)

# Test
#           1
#        /  \
#       2    3
#      / \
#     4   5
root = TreeNode(1)
root.left = TreeNode(2, TreeNode(4), TreeNode(5))
root.right = TreeNode(3)

print(level_order(root)) # [[1], [2, 3], [4, 5]]
print(inorder(root))    # [4, 2, 5, 1, 3]

```

5. Dynamic Programming (DP)

Gunakan saat: Soal minta "cara terbanyak", "minimum/maximum", atau punya overlapping subproblems.

Pola kunci:

1. Tentukan **state** (biasanya index + parameter).
2. Buat **base case**.
3. Rumus **recurrence**.
4. Optimasi dengan memoization (top-down) atau tabel (bottom-up).

```

# Fibonacci ke-n (bottom-up)
def fib(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

# Minimum coins (coin change)
def min_koin(koin, target):
    dp = [float('inf')] * (target + 1)
    dp[0] = 0
    for i in range(1, target + 1):
        for k in koin:
            if i - k >= 0:

```

```

        dp[i] = min(dp[i], dp[i - k] + 1)
    return dp[target] if dp[target] != float('inf') else -1

# Test
print(fib(10))           # 55
print(min_koin([1, 3, 4], 6)) # 2 (3+3)
print(min_koin([2], 3))    # -1

```

Tips Belajar DSA

- Kerjakan soal **bertema sama** (hanya sliding window) sampai pola terasa alami.
- Gunakan **NeetCode 150** atau **Grind 75** — bukan LeetCode acak.
- Batasi waktu: 20-30 menit per soal. Lewat? Lihat solusi, pahami, ulangi 2 hari kemudian.
- Kompleksitas waktu & ruang wajib disebut di setiap jawaban.

System Design Interview Framework (1-2-3 Method)

Untuk fresh grad, system design bukan tentang skalabilitas ribuan server. Fokus pada **fondasi dan struktur berpikir**.

Metode 1-2-3

Langkah	Durasi	Aktivitas
1. Klarifikasi	3-5 menit	Tanya kebutuhan, batasan, asumsi
2. High-Level	5-7 menit	Gambar arsitektur, komponen utama
3. Deep Dive	10-15 menit	Detail DB schema, API, trade-off

1. Klarifikasi — Tanyakan Ini

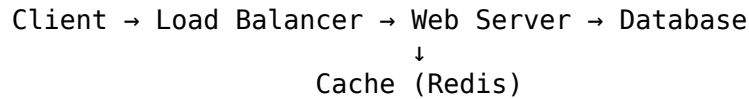
Sebelum jawab, tanya:

- "Berapa banyak pengguna?"
- "Fitur utama apa saja?"
- "Apakah perlu real-time?"
- "Prioritas availability atau consistency?"
- "Pakai database SQL atau NoSQL?"

Ini menunjukkan **cara kerja profesional** — bukan langsung tebak.

2. High-Level Architecture

Gambar (di papan tulis / whiteboard):



Sebutkan komponen dan fungsinya secara singkat.

3. Deep Dive

Pilih 1-2 area untuk di-explore detail:

- **Database schema:** tabel, kolom, relasi, index.
- **API design:** endpoint, method, request/response.
- **Data flow:** bagaimana data bergerak dari client ke database.

Contoh: Design URL Shortener

Klarifikasi:

- 1000 URL baru/hari.
- URL disimpan 1 tahun.
- Perlu tracking klik.

High-Level:

Client → POST /shorten → Web Server → DB (URLs)

Client → GET /abc123 → Web Server → Redirect 302

Deep Dive:

-- Database schema

```
CREATE TABLE urls (
  id BIGSERIAL PRIMARY KEY,
  long_url TEXT NOT NULL,
  short_code VARCHAR(7) UNIQUE NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),
  click_count INT DEFAULT 0
);
```

-- Index untuk lookup cepat

```
CREATE INDEX idx_short_code ON urls(short_code);
```

Generate short code – Base62

```
import string
```

```
BASE62 = string.digits + string.ascii_lowercase + string.ascii_uppercase
```

```
def encode(num):
    if num == 0:
        return BASE62[0]
    result = []
    while num > 0:
        result.append(BASE62[num % 62])
        num //= 62
    return ''.join(reversed(result))

print(encode(123456)) # "W7E"
```

Trade-off:

- Base62 vs hash: Base62 simpel, terjamin unik. Hash perlu cek duplikat.
- 302 redirect vs 301: 302 untuk analytics, 301 untuk cache.

Contoh: Design Chat App

Klarifikasi:

- Chat 1-on-1 (bukan grup).
- 10.000 pengguna aktif.
- Perlu status online.

High-Level:

```
Client → WebSocket → Chat Server → DB (messages)
                                ↓
                                Redis (online status)
```

Deep Dive:

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    online BOOLEAN DEFAULT FALSE
);

CREATE TABLE messages (
    id BIGSERIAL PRIMARY KEY,
    sender_id INT REFERENCES users(id),
    receiver_id INT REFERENCES users(id),
    content TEXT NOT NULL,
    sent_at TIMESTAMP DEFAULT NOW(),
    is_read BOOLEAN DEFAULT FALSE
);

CREATE INDEX idx_conversation ON messages(sender_id, receiver_id, sent_at);
```

```

# WebSocket event handler (pseudo)
async def handle_message(data):
    # data = { "to": user_id, "content": "Halo" }
    msg = {
        "from": current_user.id,
        "content": data["content"],
        "timestamp": now()
    }
    # Simpan ke DB
    await db.messages.insert(msg)
    # Kirim real-time
    await ws_manager.send_to(data["to"], msg)
    return {"status": "sent"}

```

Trade-off:

- WebSocket vs Polling: WebSocket lebih efisien untuk real-time.
- SQL vs NoSQL: SQL untuk relationship data, NoSQL untuk scalability.
- Read receipt: Update is_read butuh write tambahan — trade-off konsistensi vs performa.

Whiteboard Approach

Saat interview (online/offline), ikuti langkah ini:

1. **Tulis ulang soal** — pastikan tidak salah paham.
2. **Tanya clarifying questions** — input valid? ukuran data? output format?
3. **Buat contoh** — input → output, trace manual.
4. **Brainstorm solusi** — sebut brute force dulu, lalu optimasi.
5. **Pseudocode** — tulis logika pakai bahasa manusia.
6. **Koding** — tulis kode bersih, pakai nama variabel jelas.
7. **Test** — jalanin dengan contoh tadi.
8. **Optimasi & trade-off** — kompleksitas waktu/ruang, alternatif.

Paling penting: Think out loud. Pewawancara nilai proses berpikir, bukan jawaban akhir.

Latihan

1. **DSA Coding:** Selesaikan 3 soal berikut pakai pola di atas:
 - "Cari panjang subarray terpanjang dengan jumlah $\leq$ target" (sliding window)

- *"Balikkan linked list" (two pointers)*
- *"Cari jalur terpendek dalam grid 2D dari kiri atas ke kanan bawah" (BFS)*

Tulis kode + test case + kompleksitas waktu/ruang.

2. **System Design Mini:** Design sistem sederhana. Pilih salah satu:

- **Design Parking Lot** — kendaraan masuk/keluar, hitung biaya, cek slot kosong.
- **Design Todo App** — CRUD task, deadline, kategori, filter.

Ikuti metode 1-2-3. Tulis klarifikasi, high-level, deep dive (schema + API + flow).

3. **Whiteboard Simulasi:** Ambil soal LeetCode mudah (Easy). Rekam diri sendiri (video 10 menit) sambil:

- Baca soal keras-keras
- Tanya clarifying questions
- Tulis pseudocode
- Koding
- Test
- Analisis kompleksitas

4. **Trade-off Analysis:** Dari 2 contoh system design di atas (URL shortener & chat app), tulis 3 trade-off untuk setiap keputusan desain. Format: "Pilih [X] karena [alasan], tapi konsekuensinya [Y]."

3. Mock Interview & Portfolio Finalization

Target: Simulasi mock interview, finalisasi portfolio, strategi negosiasi & follow-up.

Mock Interview Simulation

Mock interview = latihan wawancara palsu. Tujuannya: **hilangkan gugup, asah penyampaian.**

Format Mock Interview

Sesi	Durasi	Aktivitas
Pembukaan	2 menit	Perkenalan diri
Behavioral	10 menit	2-3 pertanyaan STAR
DSA Coding	20 menit	1 soal LeetCode medium
System Design	15 menit	1 pertanyaan□□
Portfolio Walkthrough	8 menit	Jelaskan proyek terbaik
Q&A / Penutup	5 menit	Pertanyaan balik ke interviewer

Contoh Pertanyaan Mock Interview

Behavioral:

1. "Ceritakan tentang dirimu dalam 30 detik."
2. "Proyek apa yang paling menantang dan kenapa?"
3. "Pernah gagal deliver sesuatu? Apa yang terjadi?"
4. "Bagaimana cara kamu belajar teknologi baru?"
5. "Kenapa kamu tertarik sama posisi ini?"

DSA Coding:

1. "Cari pasangan angka yang jumlahnya target" (Two Pointers)
2. "Valid palindrome" (Two Pointers)
3. "Reverse linked list" (Iterative/Recursive)
4. "Binary search" (Divide & Conquer)
5. "Maximum subarray sum" (Kadane's Algorithm)

System Design (Fresh Grad):

1. "Design URL shortener"
2. "Design chat app sederhana"
3. "Design parking lot"
4. "Design todo API"
5. "Design rating/review system"

Evaluasi Mock Interview

Kriteria	Skor 1-5	Catatan
Clarity of speech		
STAR structure		
Coding correctness		
Think out loud		
Time management		
Interaction with interviewer		

Cara pakai: Minta teman jadi interviewer. Rekam video. Evaluasi sendiri pakai tabel di atas. Ulangi sampai skor minimal 4 di semua kategori.

Coding Challenge Strategy

Flow Pengerjaan (10 Langkah)

1. Baca soal 2x – pahami input/output
2. Tanya clarifying questions
3. Buat contoh kasus (input → output)
4. Sebut brute force + kompleksitas
5. Brainstorm optimasi
6. Tulis pseudocode + minta feedback
7. Koding – syntax rapi, variable jelas
8. Test dengan contoh tadi
9. Test edge cases (kosong, besar, negatif)
10. Sebut kompleksitas + trade-off

Common Mistakes

Kesalahan	Solusi
Langsung coding tanpa klarifikasi	Tanya dulu: "Input bisa kosong?"
Diam 5 menit	Paksa diri ngomong: "Saya lagi mikir..."
Lupa test case	Siapkan 2-3 contoh sebelum nulis
Syntax error di live coding	Tulis pelan, test tiap baris
Tidak sebut kompleksitas	Akhiri dengan "O(n) time, O(1) space"

Contoh Live Coding Walkthrough

Soal: "Balikkan string. Input: 'hello' → Output: 'olleh'"

"Oke, saya baca soal. Input string, output string terbalik.

Clarify: Apakah boleh pakai built-in reverse? Boleh? Baik, saya coba dua pendekatan

Pertama yang simpel: balik manual pakai two pointers.

Dua pointer – satu di kiri, satu di kanan, swap terus sampe ketemu.

Pseudocode:

```
left = 0, right = len - 1
while left < right:
    swap(s[left], s[right])
    left++, right--
```

```
Koding: (nulis)
def balik_string(s):
    arr = list(s)
    kiri, kanan = 0, len(arr) - 1
    while kiri < kanan:
        arr[kiri], arr[kanan] = arr[kanan], arr[kiri]
        kiri += 1
        kanan -= 1
    return ''.join(arr)
```

Test: 'hello' → ['h','e','l','l','o'] → swap h/o → ... → 'olleh' □

Edge case: '' → return '' □

Edge case: 'a' → 'a' □

Kompleksitas: $O(n)$ time, $O(n)$ space (karena array baru).
Bisa $O(1)$ space kalau inputnya mutable array (in-place).

Portfolio Project Walkthrough

Di interview, kamu akan diminta **jelaskan salah satu proyek** secara detail.

Struktur Walkthrough

Bagian	Durasi	Isi
Judul & tujuan	30 detik	Nama proyek, kenapa dibuat
Tech stack	30 detik	Bahasa, framework, database, deploy
Arsitektur	2 menit	Diagram + alur data
Fitur utama	1 menit	2-3 fitur dengan detail kode
Tantangan	1 menit	Bug/tantangan → solusi → hasil
Pelajaran	30 detik	Yang dipelajari, yang akan diperbaiki

Contoh Walkthrough: Task Manager API

****Judul:**** Task Manager API – REST API manajemen tugas.

****Kenapa:**** Karena saya lihat banyak teman kesulitan tracking deadline tugas sekolah.

****Tech stack:**** Node.js + Express + PostgreSQL + JWT + Railway (deploy).

****Arsitektur:**** Client → REST API → Controller → Service → Repository → DB.

Alur: Login → JWT → CRUD task dengan deadline & kategori.

****Fitur utama:****

1. Autentikasi JWT – register/login, setiap request divalidasi.
2. CRUD task – create, read, update, delete dengan filter status & kategori.
3. Deadline reminder – cron job kirim notifikasi H-1 deadline.
(point plus: ini fitur ekstra yang tidak diminta)

****Tantangan:**** Waktu itu saya pakai bcrypt untuk hash password, tapi lupa handle error saat email sudah terdaftar. Akhirnya nambah unique constraint di DB + error handling middleware.

****Pelajaran:**** Selalu validasi input sebelum simpan ke DB. Ke depannya saya mau tambah unit test dengan Jest.

Portfolio Final Checklist

- ☐ Semua README proyek lengkap (judul, stack, fitur, cara jalanin, screenshot)
- ☐ GitHub rapi — repositori fork/value dicleaned up
- ☐ Commit message pakai format: feat:, fix:, docs:
- ☐ .gitignore — tidak ada node_modules/, .env, __pycache__/
tercommit
- ☐ Live demo bisa diakses (Vercel/Netlify/Railway)
- ☐ 3 proyek dengan stack berbeda (misal: REST API + React app + mobile)
- ☐ Project walkthrough siap diceritakan tanpa lihat catatan

Negotiation Tips

Untuk fresh grad, gaji mungkin sudah fixed. Tapi kamu tetap bisa negosiasi **bukan-gaji**.

Yang Bisa Dinegosiasi Fresh Grad

Item	Cara Minta
Gaji	Riset gaji rata-rata (Glassdoor, LinkedIn). Minta 10-20% di atas. Alasan
Tunjangan	Transport, makan, internet — tanya apakah ada
Training	"Apakah ada budget untuk kursus/sertifikasi?"
Perangkat	Laptop, monitor — mintalah jika tidak disediakan
Remote/Fleksibel	"Berapa hari WFH per minggu?"
Magang → Full-time	"Apakah ada jalur konversi setelah magang?"

Cara Negosiasi yang Benar

"Terima kasih atas tawaran. Saya sangat antusias bergabung. Sebelum saya tanda tangan, boleh tanya – apakah ada fleksibilitas untuk [item]? Saya riset gaji rata-rata posisi ini di [kota] adalah [angka]. Saya yakin skill saya [sebut 2 skill relevan] bisa berkontribusi sesuai ekspektasi."

Jangan Lakukan Ini

❌ Jangan	✅ Lakukan
"Saya dapat tawaran lebih tinggi di perusahaan lain"	"Saya riset gaji rata-rata posisi ini..."
Minta gaji muluk tanpa data	Bawa data konkret
Terima langsung tanpa baca kontrak	Minta waktu 1-2 hari
Negosiasi lewat telepon tanpa persiapan	Siapkan poin-poin dulu

Follow-Up Strategy

Setelah Interview

Waktu	Tindakan
1 jam	Kirim thank you email ke interviewer
1 hari	Catat pertanyaan yang diajukan — evaluasi jawaban
3 hari	Jika belum ada kabar, follow-up sopan
1 minggu	Jika tidak diterima, minta feedback
2 minggu	Lanjut apply perusahaan lain

Contoh Thank You Email

Subject: Terima kasih – [Nama Posisi] – [Namamu]

Halo [Nama Interviewer],

Terima kasih atas waktu interview hari ini. Saya makin yakin kalau [Nama Perusahaan] adalah tempat yang tepat buat saya berkembang.

Saya sangat tertarik dengan [sebut sesuatu dari interview – misal: tech stack yang dipakai / budaya perusahaan].

Satu hal yang saya pikirkan setelah interview – [opsional: tambahan jawaban / koreksi].

Semoga bisa bergabung!

Salam,
[Nama]
[LinkedIn]
[No. HP]

Jika Diterima

1. Baca kontrak dengan teliti.
2. Tanyakan: gaji, tunjangan, jam kerja, probation period, aturan WFH.
3. Konfirmasi tanggal mulai.
4. Minta offer letter resmi (PDF).
5. Beri kabar ke perusahaan lain yang masih proses.

Jika Tidak Diterima

1. Minta feedback spesifik: "Skill apa yang kurang?"
2. Jangan defensif — catat, evaluasi, perbaiki.
3. Tanya: "Apakah ada rekomendasi course/buku?"
4. apply ke perusahaan lain dengan perbaikan.
5. Coba lagi 6 bulan kemudian setelah improvement.

Penting: Setiap penolakan adalah data. Gunakan untuk iterasi.

Latihan

1. **Mock Interview Roleplay:** Cari teman (online/offline). Lakukan mock interview 30 menit pakai format di atas. Rekam. Evaluasi diri pakai tabel skor. Tulis 3 hal yang perlu diperbaiki.
2. **Project Walkthrough Script:** Pilih proyek terbaik di GitHub. Tulis script walkthrough 5 menit pakai struktur di atas. Rekam video (HP/laptop). Evaluasi: apakah terdengar jelas? Apakah terlalu cepat? Perbaiki sampai natural.
3. **Negosiasi Simulasi:** Cari 3 lowongan fresh grad di LinkedIn/Jobstreet. Catat gaji yang ditawarkan. Tulis script negosiasi untuk masing-masing (minta 10-20% lebih tinggi + alasan). Baca keras-keras.

4. **Follow-Up Email Draft:** Buat 3 template email:

- Thank you email setelah interview
- Follow-up email (3 hari setelah interview, belum ada kabar)
- Feedback request email (setelah ditolak)

Personalisasi untuk perusahaan fiktif bernama "TechIndo Startup".